

Lightbeam: Beam Propagation Method Python Package

Nathan K. Long,^{1,*} Justin Vella,¹ and Barnaby Norris¹

¹*Astralis-USyd, Sydney Institute for Astronomy (SIfA),
School of Physics, University of Sydney, Camperdown, NSW, Australia*
(Dated: March 18, 2026)

INTRODUCTION

Lightbeam is a beam propagation method (BPM) Python package [1], developed by Jonathan Lin, based on a finite-difference BPM [2] with adaptive meshing [3]. BPM is a numerical technique for modelling guided optical fields by transforming the wave equation into a forward-propagating evolution equation along the longitudinal z -axis. Under the paraxial approximation, the field envelope is advanced stepwise, enabling efficient simulation of long waveguiding structures [2].

Finite-difference BPMs discretise the transverse derivatives using finite differences, where lightbeam uses the generalized Douglas scheme, resulting in the stepwise linear system solved at each propagation step. Finite-difference BPMs support nonuniform or adaptive grids for complex geometries [3], such as photonic lanterns (PLs), whose geometries vary longitudinally. Lightbeam implements a refinement factor, which is effectively used to calculate where to refine the grid meshing, as a function of the field amplitude and second derivative magnitude (lower values equals finer mesh, and vice-versa).

This report examines the performance of lightbeam, providing an overview of the lightbeam BPM, a validation of lightbeam against a commercial BPM, examines meshing parameterisation for output field convergence, and undertakes a computational time analysis. Specifically, the performance analysis implements a 19-core PL (19cPL) as the simulated optical object. The output electric fields of the 19cPL are compared with those of Keysoft's RSoft BeamPROP BPM (taken as the industry standard BPM software). The 19cPL is then used for the meshing convergence and temporal analyses, determining the 'best' meshing parameters to achieve greatest computational efficiency.

I. PHOTONIC LANTERN

PLs are photonic wavefront sensors, where an incoming electric field is focused onto a multimode fibre (MMF) at the input, then transformed into N single-mode fibres (SMFs) at the output. The N output intensities are then used as input to a neural network, which maps the non-linear relationship between the output core intensities

and a reconstruction of a wavefront correction to compensate for upstream distortion of the electric field as it propagates from an extrasolar source to the detector [4]. 19cPLs are shown for wavefront sensing experimentally at the Subaru/SCEXAO in [5, 6] and on the Shane telescope in [7].

A 19cPL optical object is constructed in lightbeam, acting as the simulation model for the performance analysis, where the 19cPL model used for simulation is designed using the properties given in Table I.

The response of the 19cPL to 17 excited LP modes is simulated, at wavelength $\lambda = 1500$ nm, which are input to the PL and propagated through it to the 19c outputs using lightbeam, where the 17 excited modes are shown in Appendix A. Each output core is given an index, which is used throughout the subsequent analyses, and is shown for the 19cPL in Figure 1.

Note that an example code for propagating 17 LP modes through a 19cPL is given in Appendix D. A more in-depth tutorial is provided at [1].

II. VALIDATION

The same 19cPL model is constructed in RSoft and lightbeam, and the same 17 excited LP modes are propagated through them, where the same meshing parameters are used for both lightbeam and RSoft (given in Table II). Note that RSoft does not have adaptive meshing, thus the adaptive meshing reference factor is set to 1 for lightbeam (representing uniform meshing).

Figure 2 shows the intensity matrices for RSoft, lightbeam, and the difference between RSoft intensities $P_{rs}(lm, c)$ and lightbeam intensities $P_{lb}(lm, c)$, $|P_{rs}(lm, c) - P_{lb}(lm, c)|$, across the 19 output cores (c) for the 17 input LP modes (lm).

It can be seen that, in general, there is a close alignment

TABLE I. Photonic lantern properties.

Parameter	Value
Output core radii	$3.25 \mu\text{m}$
Output cladding radius	$164 \mu\text{m}$
Output core spacing	$60 \mu\text{m}$
Taper ratio	10
Core refractive indices	1.4467895
Cladding refractive index	1.44
Capillary refractive index	1.4345
PL length	$40,000 \mu\text{m}$

* nathan.long@sydney.edu.au

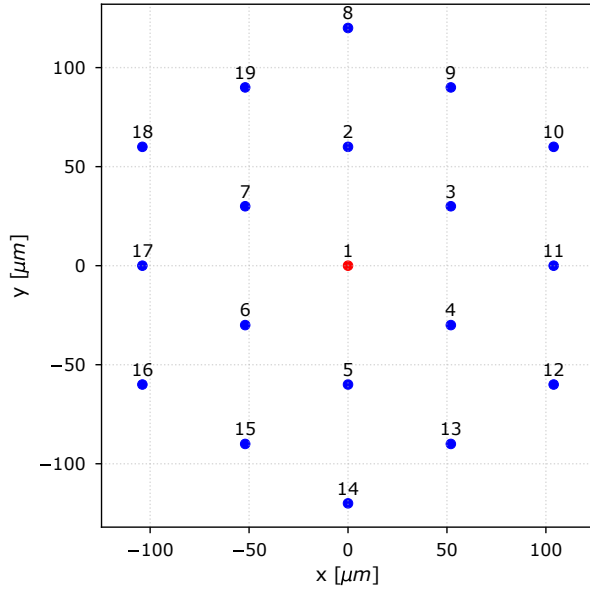


FIG. 1. 19 core photonic lantern output core indices.

between the RSoft and lightbeam intensity matrices. The greatest difference exists in the output intensity for core 1 when excited by mode LP02. As can be seen in the RSoft intensity matrix, the LP02 and LP03 modes should have similar intensities in core 1. Further work is required to determine the source of this discrepancy.

The same comparison is then undertaken in Figure 3 for the phase matrices generated by RSoft, lightbeam, and the difference between RSoft phases $\Phi_{rs}(lm, c)$ and lightbeam phases $\Phi_{lb}(lm, c)$, $|\Phi_{rs}(lm, c) - \Phi_{lb}(lm, c)|$, across the 19 output cores for the 17 input LP modes. While the intensity matrices are approximately aligned, there is a clear distinction between the RSoft and lightbeam phase matrices. There is a uniform phase offset between RSoft and lightbeam for a majority of output cores for each excited mode, which could be attributed to the particular implementation of BPMs.

However, there exists patterns of non-uniform phase differences in certain each core, which can be separated into different groups of cores. Firstly, the central core 1 has a different phase offset across all of the excited modes. Secondly, for the excited modes LP01, LP02, LP21a, LP21b, LP41a, and LP41b, there is a different uniform phase offset for the cores 2-7, which represent the inner ring of cores. Thirdly, there are further slight non-

uniform phase offsets for many of the excited modes, where the slight offset is present between odd and even cores > 7 , representing the closer and further cores in the outer ring. These results indicate that the electric field propagation has a different dependence on the radial distance from the centre, as compared to RSoft. Finally, there are isolated cores with greater phase offsets, not dependent on their radial distance from the centre.

To understand the cause of non-uniform phase offsets in the isolated cores, the intensities of the output cores for each of the excited modes are plotted in Figure 4, where the non-uniform phase offsets in the isolated cores are plotted as black-ringed red dots.

By analysing the Figures 2 and 4, it can be seen that all of the non-uniform phase offsets are associated with low or $\sim$ zero intensity in that core, all having intensities below ~ 0.012 . Taking the LP11b mode as an example, the non-uniform phase offsets are present in cores 1, 2, 5, 8, 14. By referencing this with the core locations in Figure 1, the input LP11b intensity profile in Figure 7, and the 19cPL output intensity profile for the LP11b profile in Figure 8, the near-zero intensity profile for the excited LP11b mode can be logically seen, where there is $\sim$ zero intensity in the cores 1, 2, 5, 8, 14. The same pattern can be seen across most of the rest of the non-uniform phase offsets, which indicates that the phase offset information in these core is unrealistic and not relevant to the simulation results.

Therefore, we consider the uniform phase offsets, as a function of radial distance, between RSoft and lightbeam for each excited mode to be associated with variational implementations of a finite-difference BPM, where low intensities result in unrealistic phase offsets, and the uniform phase offsets do not affect the relative phase difference between the output cores. As such, we consider lightbeam to be validated against the RSoft results, at least for the 19cPL model.

III. MESHING CONVERGENCE

A range of meshing parameters are simulated to assess the resolution required for lightbeam to converge on a solution for the output electric fields from the 19cPL simulations. Specifically, a range of values for the transverse resolution ds , longitudinal resolution dz , and refinement factor rf are simulated.

A preliminary analysis is undertaken by calculating the normalised Euclidean distance ϵ between the output core intensity matrices,

$$\epsilon = \frac{\|P_{ref}(lm, c) - P_{lb}(lm, c)\|_F}{\|P_{ref}(lm, c)\|_F}, \quad (1)$$

where the meshing parameters defined in Table II are considered the ‘reference parameters’ (termed $P_{ref}(lm, c)$) for comparison with all other meshing parameters tested.

TABLE II. Reference meshing parameters.

Parameter	Value
Transverse diameter	349 μm
Perfectly matched layers	10
Transverse resolution	0.25 μm
Longitudinal resolution	2 μm
Adaptive meshing reference value	1

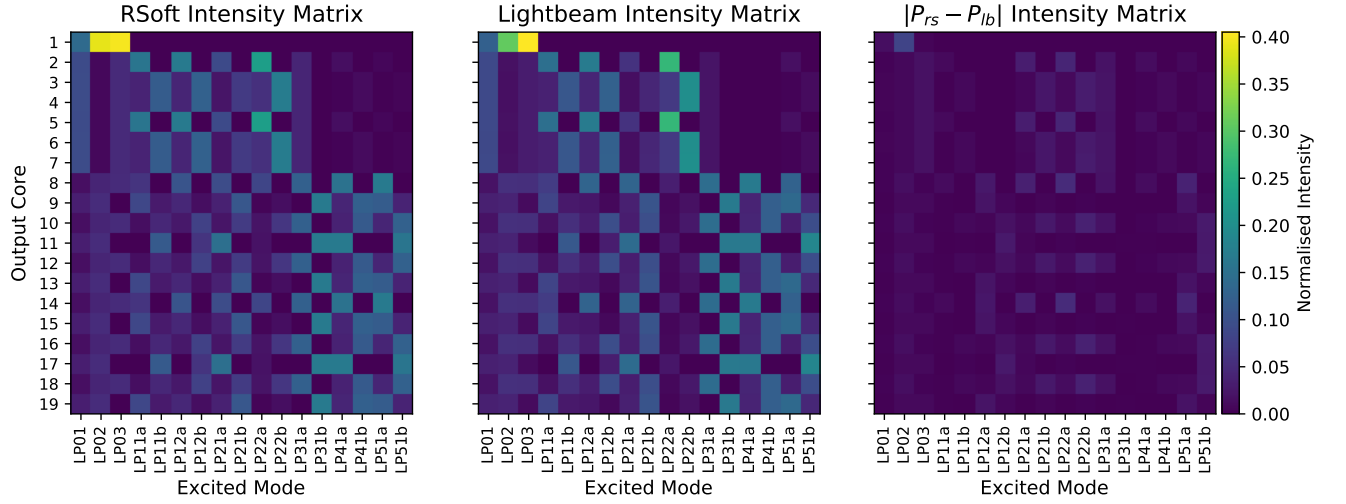


FIG. 2. Lightbeam RSoft intensity matrix comparison.

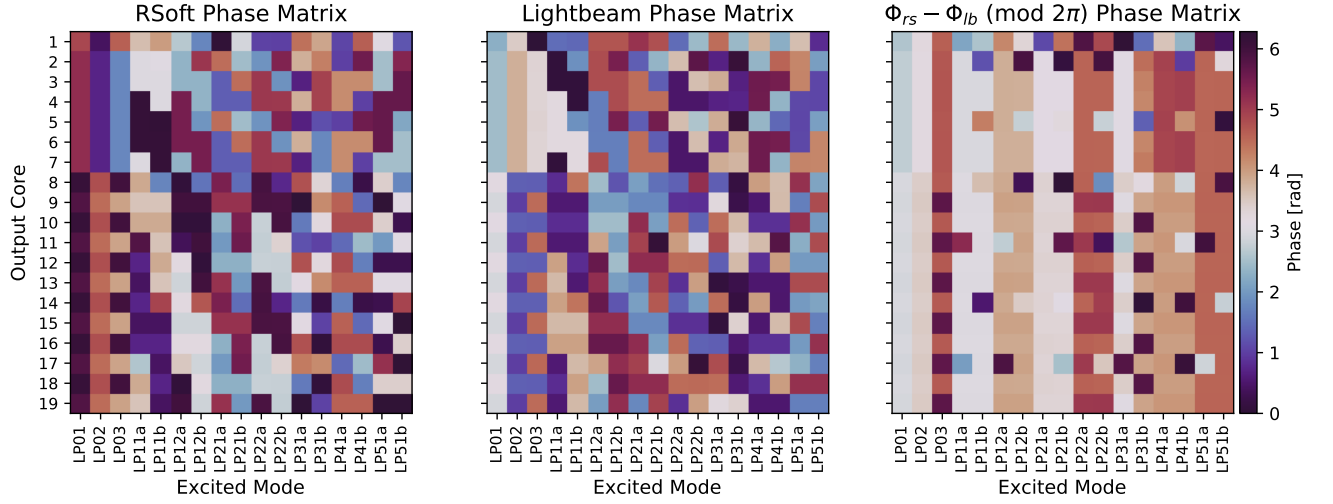


FIG. 3. Lightbeam RSoft phase matrix comparison.

Figure 5 shows the normalised Euclidean distance for range of the meshing parameters tested, where the reference matrix is outlined in red.

Considering the range of the parameters ds , dz , and rf tested, ds has the greatest effect on the simulation performance, where the greatest distance from $P_{ref}(lm, c)$ is found when increasing ds from $0.25 \mu\text{m}$ to $0.50 \mu\text{m}$. It is shown that dz then has the second greatest effect on the simulation performance, where the distance increases as dz is increased to $4 \mu\text{m}$ and again to $6 \mu\text{m}$ from $2 \mu\text{m}$. Note that lightbeam was unable to propagate light for $ds = 0.50 \mu\text{m}$ and $dz = 2 \mu\text{m}$, due to mismatching array shapes as the light is propagated longitudinally. Interestingly, rf appears to have relatively little effect on the simulation performance, at least for the magnitudes tested (Lin recommends using lower refinement factors, however, computational time increases dramatically).

Another comparison of the meshing convergence is un-

dertaken by plotting the phases (shown in Appendix C) across all cores, for each excited mode, for each of the combinations of meshing parameters. Across all of the excited modes, it appears that the phases remain relatively constant for the meshing combinations with $ds = 0.25 \mu\text{m}$ and $dz \leq 4 \mu\text{m}$, before diverging for all of the cases with $ds = 0.50 \mu\text{m}$ or $dz = 6 \mu\text{m}$. The coarsest meshing, which converges on approximately same result, is thus $ds = 0.25 \mu\text{m}$ and $dz = 4 \mu\text{m}$, and $rf = 1$.

IV. TEMPORAL ANALYSIS

To understand the time required to propagate a single excited mode through the 19cPL using lightbeam, the computational time is recorded for the same range of meshing parameters tested in Section III, as shown in Figure 6, where the number of available threads is restricted

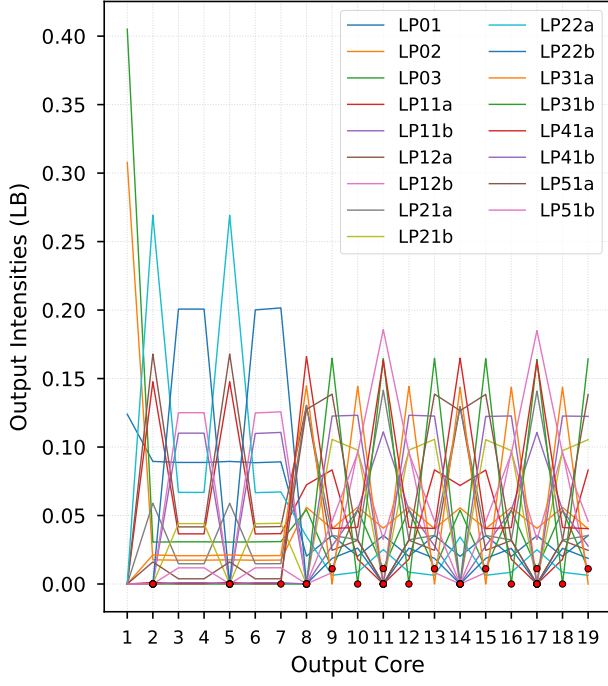


FIG. 4. Output intensities in each core for each excited mode, where the non-uniform phase offsets are plotted as red dots.

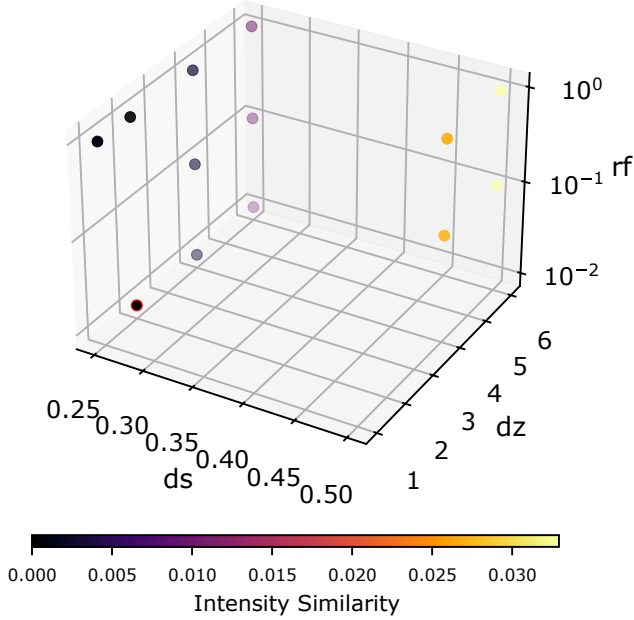


FIG. 5. Lightbeam meshing convergence.

to eight.

The fastest simulation time is ~ 17 mins for $ds = 0.50 \mu\text{m}$, $dz = 6 \mu\text{m}$ and $rf = 1$, as well as $ds = 0.50 \mu\text{m}$, $dz = 6 \mu\text{m}$ and $rf = 0.1$. The longest simulation time is for $ds = 0.25 \mu\text{m}$, $dz = 1 \mu\text{m}$, and $rf = 1$, at 355 mins.

By comparing the temporal analysis with the meshing

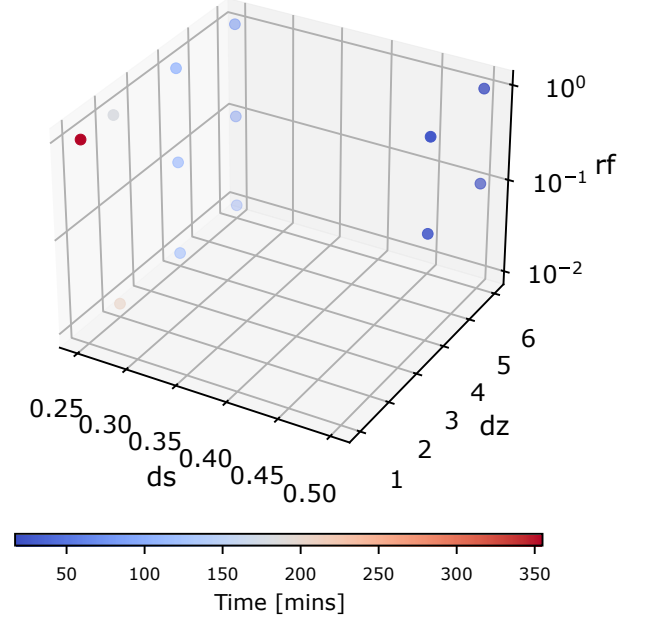


FIG. 6. Lightbeam computational temporal analysis.

convergence, the best combination of meshing parameters to use is $ds = 0.25 \mu\text{m}$ and $dz = 4 \mu\text{m}$, where the simulation times for $rf = 0.01$, $rf = 0.1$, and $rf = 1$ are all approximately 100 mins (using eight threads), given that the simulation outputs have converged, and the computational time is minimised. It is worth noting that, for the same meshing combination, RSoft simulates a single-core propagation in *** mins, highlighting an advantage of RSoft over lightbeam.

An analysis of the effects of the simulation time versus the number of threads is undertaken for further minimisation of the computational resources required for lightbeam. Table III shows the simulations times for a range of available threads, for both the reference meshing combination ($ds = 0.25 \mu\text{m}$, $dz = 2 \mu\text{m}$, and $rf = 0.01$) and the 'best' meshing combination ($ds = 0.25 \mu\text{m}$, $dz = 4 \mu\text{m}$, and $rf = 1$).

As can be seen, two threads minimise the simulation time required to propagate a single excited mode through a 19cPL, where the simulation time reduces from 204 mins to 163 mins when eight threads is reduced to two threads for the reference meshing combination. Similarly, the simulation time is reduced from 102 mins

TABLE III. Simulation time versus number of threads for a single excited mode propagated through a 19cPL.

# Threads	Reference	Best
1	170 mins	87 mins
2	163 mins	83 mins
4	195 mins	89 mins
8	204 mins	102 mins
16	210 mins	103 mins

to 83 mins for the best meshing combination, when the number of threads is reduced from eight to two, offering the minimal simulation time for the meshing parameters tested.

V. FUTURE WORK

One major future objective could be to optimise lightbeam for faster simulation time, given that it is open source, such that licensing fees for RSoft could be mitigated when simulating wave propagation through weakly-guided waveguides.

There are also several further minor meshing parameters which could be varied to understand if further reduction in computational times can be achieved, included the number of longitudinal steps to simulate before remesh-

ing.

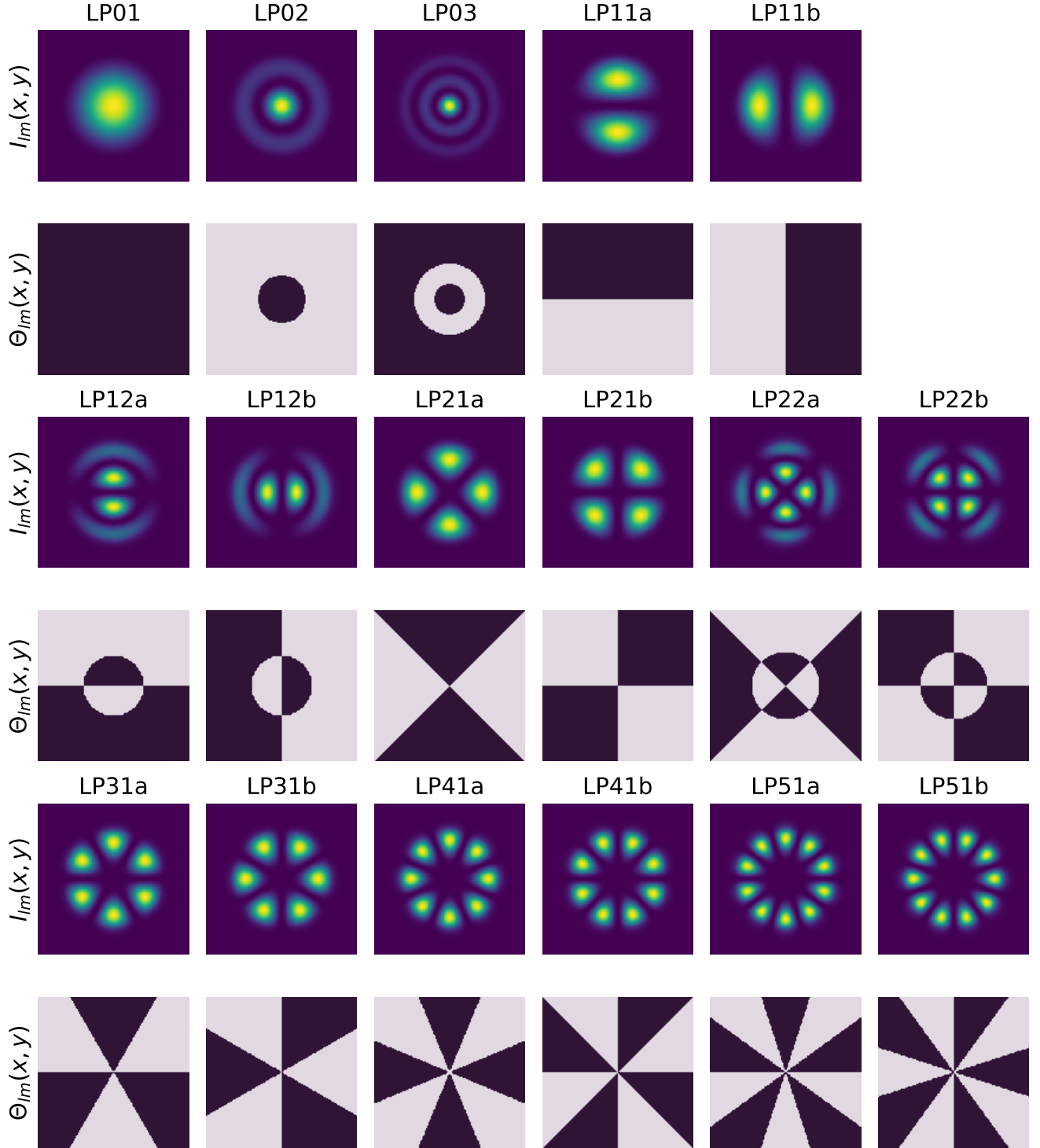
The current implementation of lightbeam is not able to propagate light for most meshing values, as there exists an array mismatch along the stepwise longitudinal propagation. Modifying lightbeam to accept any meshing values could be useful for further meshing optimisation.

VI. CONCLUSION

Lightbeam has been shown as an alternative to the commercial RSoft BPM software, with simulated performance matching that of RSoft closely, notwithstanding the uniform phase offsets as a function of radial distance. While the simulations times are greater for lightbeam, if BPM simulation is not time-critical, lightbeam offers a free solution, without limits on the number of simultaneous users, without requiring commercial licensing fees.

-
- [1] lightbeam, <https://github.com/SAIL-Labs/lightbeam> (2026), accessed: 2026-03-18.
 - [2] G. L. Pedrola, *Beam propagation method for design of optical waveguide devices* (John Wiley & Sons, 2015).
 - [3] J. Shibayama, K. Matsubara, M. Sekiguchi, J. Yamauchi, and H. Nakano, Efficient nonuniform schemes for paraxial and wide-angle finite-difference beam propagation methods, *Journal of Lightwave Technology* **17**, 677 (1999).
 - [4] B. R. Norris, J. Wei, C. H. Betters, A. Wong, and S. G. Leon-Saval, An all-photonic focal-plane wavefront sensor, *Nature Communications* **11**, 5335 (2020).
 - [5] J. W. Lin, M. P. Fitzgerald, Y. Xin, Y. J. Kim, O. Guyon, B. Norris, C. Betters, S. Leon-Saval, K. Ahn, V. Deo, *et al.*, Real-time experimental demonstrations of a photonic lantern wave-front sensor, *The Astrophysical Journal Letters* **959**, L34 (2023).
 - [6] J. Lin, M. P. Fitzgerald, Y. Xin, Y. Jung Kim, O. Guyon, B. Norris, C. Betters, S. Leon-Saval, K. Ahn, V. Deo, *et al.*, Experimental and on-sky demonstration of spectrally dispersed wavefront sensing using a photonic lantern, *Optics Letters* **50**, 2780 (2025).
 - [7] A. R. Sengupta, J. Diaz, M. DeMartino, R. Jensen-Clem, S. Cetre, E. Gates, K. Bundy, D. Dillon, P. Hinz, M. Salama, *et al.*, On-sky demonstration of second-stage wave-front control with a photonic lantern, *The Astronomical Journal* **171**, 65 (2026).

Appendix A: LP Modes

FIG. 7. Excited LP mode transverse intensities $I_{lm}(x, y)$ and phases $\Theta_{lm}(x, y)$ (for an arbitrary geometry).

Appendix B: 19 Core Photonic Lantern Outputs

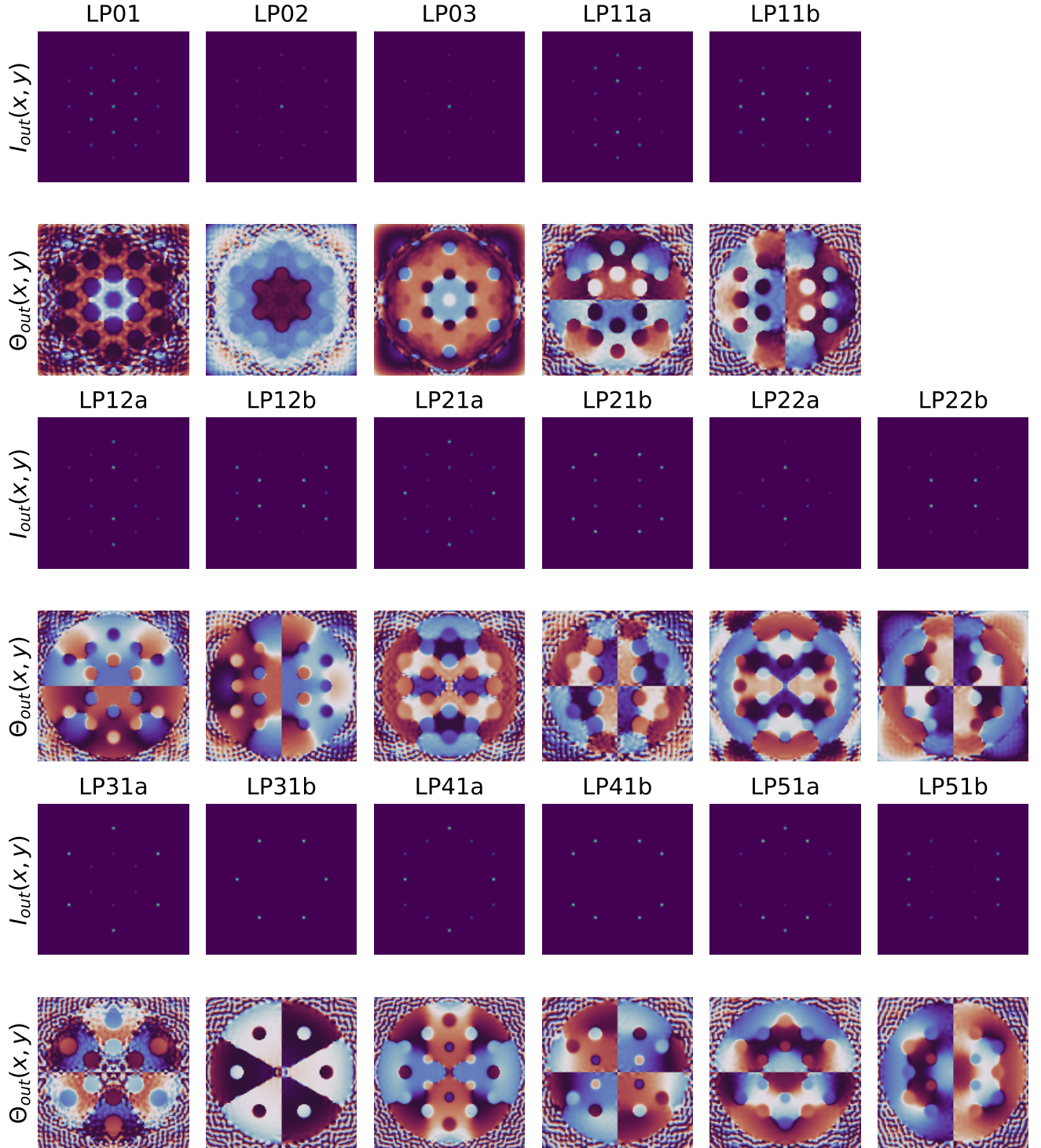


FIG. 8. 19-core photonic lantern output transverse intensities $I_{out}(x, y)$ and phases $\Theta_{out}(x, y)$ (for the parameters defined in Tables I and II).

Appendix C: Phase Meshing Convergence

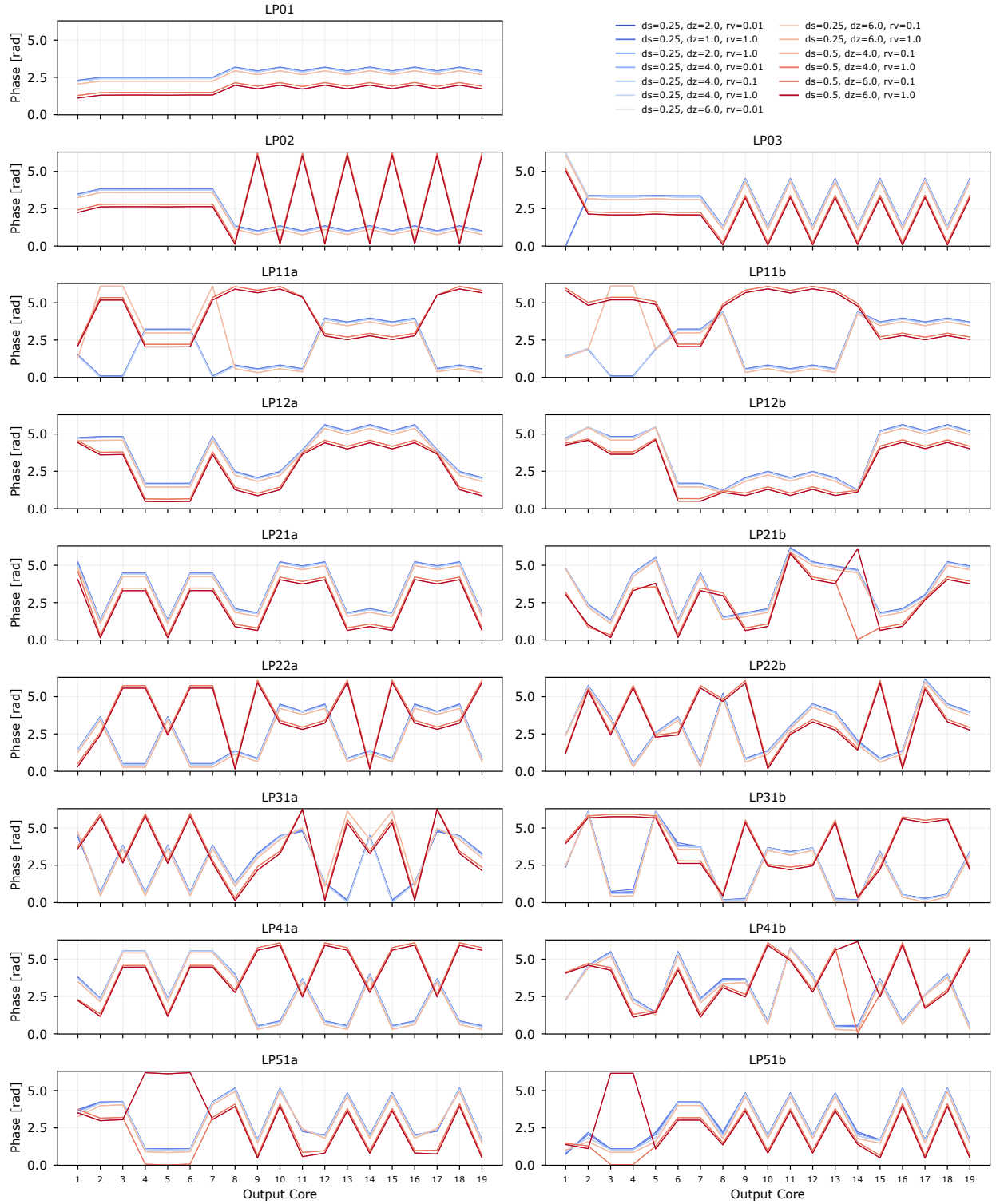


FIG. 9. Phases for each excited mode, output core, and meshing parameters.

Appendix D: Lightbeam 19cPL Example Code

```

1  %%#####
2  '''
3  Propagates 17 LP modes through a 19-core a photonic lantern using lightbeam
4
5  '''
6
7  %%#####
8  ### Limit Number of Cores
9
10 import os
11
12 # limit core usage
13 os.environ["OMP_NUM_THREADS"] = "2"
14 os.environ["OPENBLAS_NUM_THREADS"] = "2"
15 os.environ["MKL_NUM_THREADS"] = "2"
16 os.environ["NUMEXPR_NUM_THREADS"] = "2"
17 os.environ["VECLIB_MAXIMUM_THREADS"] = "2"
18 print("Thread limits set.")
19
20 %%#####
21 ### Import Libraries and Modules
22
23 import numpy as np
24 import h5py
25 import matplotlib.pyplot as plt
26
27 import lightbeam
28 import lightbeam.optics as optics
29 from lightbeam import LPmodes
30 from lightbeam.mesh import RectMesh3D
31 from lightbeam.misc import normalize, unweight_field
32 from lightbeam.prop import Prop3D
33
34
35 %%#####
36 ### Code example: 19 port lantern ###
37
38 ## Set Parameters and Variables
39
40 wl = 1.5 # wavelength [um]
41
42 #### Mesh ####
43 xw = 349 # [um]
44 yw = 349 # [um]
45 z_len = 40000 # [um]
46
47 num_PML = 10 # perfectly matched layers
48
49 ds = 0.25 # [um]
50 dz = 4 # [um]
51
52 ## Adaptive meshing refinement ratio
53 ref_val = 1
54
55 ## Create Mesh
56 _mesh = RectMesh3D(xw, yw, z_len,
57                   ds, dz, num_PML)
58
59 x_grid, y_grid = _mesh.xg[num_PML:-num_PML, num_PML:-num_PML], \
60                 _mesh.yg[num_PML:-num_PML, num_PML:-num_PML]

```

```

61
62 ## Set the final cross-sectional scale
63 taper_ratio = 10
64
65 ## Output Radii ##
66 r_core_1_out = 3.25 # [um]
67 r_core_2_out = 3.25 # [um]
68 r_clad_out = 164 # [um]
69
70 core_spacing_out = 60 # [um]
71
72 ## Input Radii ##
73 r_core_1 = r_core_1_out/taper_ratio # central science core radius [um]
74 r_core_2 = r_core_2_out/taper_ratio # outer AO cores radii [um]
75 r_clad = r_clad_out/taper_ratio # cladding radius [um]
76
77 # spacing between cores
78 core_spacing = core_spacing_out/taper_ratio # [um]
79
80 ## Refractive Indices ##
81 n_core = 1.4467895
82 n_clad = 1.44
83 n_cap = 1.4345
84
85
86 %% Create Lantern Object
87
88 pl_19 = optics.lant19(r_core_2,
89                       r_clad,
90                       n_core,
91                       n_clad,
92                       n_cap,
93                       core_spacing,
94                       z_len,
95                       z_offset=0,
96                       scale_func=None,
97                       final_scale=taper_ratio)
98
99
100
101 %%#####
102 ### Initialise
103
104 ## Set Modes Manually
105 modes = [[0, 1],
106          [0, 2],
107          [0, 3],
108          [1, 1],
109          [1, 1],
110          [1, 2],
111          [1, 2],
112          [2, 1],
113          [2, 1],
114          [2, 2],
115          [2, 2],
116          [3, 1],
117          [3, 1],
118          [3, 2],
119          [3, 2],
120          [4, 1],
121          [4, 1],
122          [5, 1],
123          [5, 1]]
124

```

```

125
126 n_modes = len(modes)
127
128 modes_flip = ['cos', 'sin']
129
130 ## Initialise Arrays to Store Data
131 u0 = []
132 U_out_weighted_array = []
133 U_out_weights_array = []
134 U_out_array = []
135
136 %% File Names
137
138 f_path = 'Results/'
139 f_suffix = f'ds={ds}_dz={dz}_rv={ref_val}_xyw={xw}_zlen={z_len}_tr={taper_ratio}'
140
141
142 %%#####
143 ### Loop Through Modes
144
145 ## Loop Through Propagation Modes
146 for i in range(n_modes):
147
148     ## Generate LP fields
149     if modes[i][0] == 0:
150         u0 = normalize(LPmodes.lpfield(x_grid, y_grid,
151                                         modes[i][0],
152                                         modes[i][1],
153                                         r_clad,
154                                         wl,
155                                         n_core,
156                                         n_clad))
157
158         ab = ''
159
160     else:
161         if i % 2 == 1:
162             u0 = normalize(LPmodes.lpfield_fixed(x_grid, y_grid,
163                                                  modes[i][0],
164                                                  modes[i][1],
165                                                  r_clad,
166                                                  wl,
167                                                  n_core,
168                                                  n_clad,
169                                                  which=modes_flip[0]))
170
171             ab = 'a'
172
173         else:
174             u0 = normalize(LPmodes.lpfield_fixed(x_grid, y_grid,
175                                                  modes[i][0],
176                                                  modes[i][1],
177                                                  r_clad,
178                                                  wl,
179                                                  n_core,
180                                                  n_clad,
181                                                  which=modes_flip[1]))
182
183             ab = 'b'
184
185
186
187
188

```

```

189     #### propagation ####
190     print("Propagating core:", i)
191     prop = Prop3D(wl,          # wavelength
192                  _mesh,       # mesh
193                  pl_19,        # optical system
194                  n_clad)       # cladding refractive index
195
196
197
198     ## prop2end
199     u_out_weighted, u_out_2, u_out_weights = prop.prop2end(u0,
200                                                            ref_val=ref_val,
201                                                            remesh_every=50)
202
203
204     ## Calculate Normalized Output Field (preserving normalized power)
205     u_out = unweight_field(u_out_weighted, u_out_weights)
206
207     print("Output power check:", np.sum(np.abs(u_out)**2))
208
209
210     ## Append output to list
211     U_out_weighted_array.append(u_out_weighted)
212     U_out_weights_array.append(u_out_weights)
213     U_out_array.append(u_out)
214
215
216     ## Plot Output Fields
217     [fig2, axs2] = plt.subplots(1, 2,
218                                sharex=True,
219                                sharey=True,
220                                tight_layout=True,
221                                figsize=(8, 4))
222     axs2[0].set_title(f'Output Intensity: LP{modes[i][0], modes[i][1]}{ab}',
223                      fontsize=14)
224     axs2[1].set_title(f'Output Phase: LP{modes[i][0], modes[i][1]}{ab}',
225                      fontsize=14)
226     im2 = axs2[0].imshow(np.abs(u_out)**2,
227                           cmap='viridis',
228                           origin='lower',
229                           label='b'))
230     im2 = axs2[1].imshow(np.angle(u_out),
231                           cmap='twilight',
232                           origin='lower',
233                           label='b'))
234     plt.tight_layout()
235     plt.show()
236
237     plt.close(fig2)
238
239
240     %% Covert to Arrays
241     U_out_weighted_array = np.array(U_out_weighted_array)
242     U_out_array = np.array(U_out_array)
243     U_out_weights_array = np.array(U_out_weights_array)
244
245
246     %%#####
247     ### Save Outputs
248
249     print("Saving...")
250
251
252

```

```
253 U_out_weighted_data = h5py.File(f_path+'U_out_weighted_array_'+f_suffix+'.h5', 'w')
254 U_out_weighted_data.create_dataset('U_out_weighted', data=U_out_weighted_array)
255 U_out_weighted_data.close()
256
257 U_out_data = h5py.File(f_path+'U_out_array_'+f_suffix+'.h5', 'w')
258 U_out_data.create_dataset('U_out', data=U_out_array)
259 U_out_data.close()
260
261 U_out_weights_data = h5py.File(f_path+'U_out_weights_array_'+f_suffix+'.h5', 'w')
262 U_out_weights_data.create_dataset('U_out_weights', data=U_out_weights_array)
263 U_out_weights_data.close()
```